

ATLAS LMS — DESIGN SYSTEM v1

Phase 0 + Phase 1A + Phase 1B

Technology: Next.js · TypeScript · Tailwind · shadcn/ui

Status: Implementation-ready design-system specification

Scope: Visual system, reusable UI patterns, component conventions, token architecture, and frontend composition rules for approved Wireframes v1 only.

0. Binding Contract

This Design System v1 sits entirely on top of the approved Atlas LMS artifacts.

It does not create new screens, APIs, permissions, workflows, entities, roles, routes, journeys, business rules, entitlement keys, or platform capabilities.

Screen Inventory v1 remains the authoritative list of screens.

Wireframes v1 remains the authoritative source for page structure, actions, entry sources, exit destinations, states, and navigation.

API Inventory v1 remains the authoritative API surface.

Permission Matrix v1 remains the authorization source of truth.

User Flows & Journey Maps v1 remains the traversal source of truth.

This document defines only how the already-approved surfaces should look, compose, scale, theme, and be implemented in a consistent frontend system.

1. Design Principles

1.1 White-label first

Every visible brand expression must come from tenant branding, theme tokens, logo assets, or platform-safe defaults.

Do not hardcode:

- FundedBeyond name
- FundedBeyond colors
- Tenant logo
- Tenant domain
- Tenant copy
- Tenant-specific labels
- Tenant-specific icons
- Tenant-specific background art

FundedBeyond Academy is Tenant #1 configuration, not a separate design fork.

1.2 Gate-before-render

Protected UI must not visually render before the route gate resolves:

1. Host resolution
2. Tenant state gate
3. Authentication
4. ACTIVE membership gate
5. Entitlement gate, where applicable
6. Permission check
7. Ownership or relationship resolution, where applicable

Before these pass, show only safe shell-level loading states. Never flash tenant data, protected nav, another tenant's branding, hidden actions, or cross-tenant resource names.

1.3 One platform, multiple planes

The design system supports separate visual planes:

Plane	UI Personality	Shell
Public	Branded, conversion-focused, minimal	Public Site Shell
Learner	Motivational, progress-focused, low friction	Learner App Shell
Instructor	Productive, builder-focused, structured	Studio Shell
Moderator	Queue-focused, evidence-first	Moderation Shell
Tenant Admin	Operational, configuration-first	Admin Console Shell
Platform Admin	Restricted, audit-heavy, neutral	Platform Console Shell

Platform UI must feel intentionally separate from tenant UI.

1.4 Clarity over decoration

Atlas is a learning infrastructure product, not a decorative landing-page template system. Visual design must support:

- Fast decision-making
- Clear ownership of actions
- Obvious permission state
- Low-friction learning
- High-trust admin operations
- Auditability
- Accessibility

1.5 Progressive complexity

Learners see the next best action first.

Instructors see creation and review status first.

Moderators see queue priority and evidence first.

Tenant admins see setup health and operational risk first.

Platform admins see tenant state, reason scope, and audit context first.

1.6 Human gate visibility

Review, publish, issue, moderate, revoke, and approval-related states must be explicit. Buttons must say what actually happens:

- Use “Submit for review” when the artifact enters review.
- Use “Approve and publish” only where approval actually publishes.
- Use “Issue certificate” only where the permission and workflow allow issuance.
- Use “Revoke” only with confirmation and audit context.

1.7 No financial-advice framing

Readiness, diagnostic, competency, and CTA components must frame outputs as educational readiness signals, not financial advice, guaranteed funding, guaranteed passing, or guaranteed profitability.

2. Information Density Rules

2.1 Density levels

Atlas uses three density modes.

Density	Used In	Characteristics
Comfortable	Learner, public, diagnostic, swipe	Larger spacing, fewer columns, single next action
Standard	Instructor, admin forms, detail pages	Balanced spacing, clear sections
Compact	Tables, queues, platform console, audit logs	Denser rows, tighter controls, persistent metadata

2.2 Learner density

Learner screens must avoid enterprise-dashboard overload.

Default learner screens should prioritize:

1. Current stage or progress
2. One primary action
3. Recommended lesson/practice/assessment

4. Secondary widgets
5. Historical data

Do not show more than three competing CTAs in the primary viewport.

2.3 Control-plane density

Studio, moderation, tenant admin, and platform screens may use denser layouts but must preserve:

- Clear page title
- Primary guarded action
- Filter/search area
- Main table/editor/queue region
- Context panel or metadata panel
- Explicit empty/loading/error states

2.4 Table density

Default table row height:

- Comfortable table: 56px
- Standard table: 48px
- Compact admin/platform table: 40px minimum

Never reduce below 40px because row actions, keyboard focus, and touch targets become unsafe.

2.5 Status density

Status badges must be short and machine-state aligned.

Examples:

- Draft
- In Review
- Published
- Suspended
- Archived
- Active
- Invited
- Removed
- Pending
- Revoked
- Failed
- Completed

Do not invent marketing status labels that do not map to approved state semantics.

3. Layout System

3.1 Shell layouts

Atlas has four structural shell families.

Public Site Shell

Composition:

1. Tenant-branded header
2. Public main content region
3. CTA/form/result/verification region
4. Footer/legal/support region

Reusable components:

- `PublicShell`
- `PublicHeader`
- `TenantLogo`
- `PublicNav`
- `PublicFooter`
- `PublicMain`
- `PublicHero`
- `PublicTrustBlock`
- `PublicLegalFooter`

Learner App Shell

Composition:

1. Tenant header
2. Search, notifications, profile
3. Learner navigation
4. Main content region
5. Contextual action area

Reusable components:

- `LearnerShell`
- `LearnerHeader`
- `LearnerSidebar`
- `LearnerMobileNav`
- `LearnerMain`
- `NextBestActionCard`
- `ProgressSummaryCard`
- `ReadinessBandCard`
- `StreakWidget`

Control Shell

Used for Studio, Moderation, and Tenant Admin.

Composition:

1. Control-plane header
2. Page title and guarded primary action
3. Role navigation
4. Main table/editor/detail/queue region
5. Secondary panels
6. Validation/history/confirmation areas

Reusable components:

- ControlShell
- ControlHeader
- ControlSidebar
- ControlMain
- ControlPageHeader
- GuardedPrimaryAction
- DetailPanel
- AuditInlinePanel
- ReviewStatusPanel

Platform Console Shell

Composition:

1. Platform header
2. MFA/session status
3. Reason context
4. Platform navigation
5. Cross-tenant table/detail region
6. Reason-bound lifecycle/support/audit actions

Reusable components:

- PlatformShell
- PlatformHeader
- PlatformReasonBanner
- PlatformSidebar
- PlatformMain
- TenantStateBadge
- PlatformAuditPanel

3.2 Page anatomy

Every page must follow this structure:

```
<Shell>
  <PageGate>
    <PageHeader />
    <PageBody>
      <PrimaryRegion />
      <SecondaryRegion />
    </PageBody>
  </PageGate>
</Shell>
```

3.3 Page header standard

Each authenticated page must have:

- Title
- Optional subtitle
- Breadcrumb only for deep editor/detail pages
- Status badge where resource state matters
- Primary action, if permitted
- Secondary actions in overflow menu
- Optional audit/history indicator for sensitive resources

3.4 Detail page standard

Detail pages use:

- Header with title/status/actions
- Main content column
- Right-side metadata panel on desktop
- Collapsed metadata accordion on mobile

Used for:

- Course detail
- Assessment detail
- Attempt detail
- Member detail
- Moderation case detail
- Tenant detail
- Certificate detail
- Workflow review detail

3.5 Builder page standard

Builder screens use:

- Left outline/navigation panel
- Center editor canvas
- Right configuration/validation panel

- Sticky save/review state
- Inline errors
- Review submission action

Used for:

- Course builder
- Lesson editor
- Learning path builder
- Item/question builder
- Assessment builder
- Certificate template editor
- Branding/theme editor
- Notification template editor

3.6 Queue page standard

Queue screens use:

- Queue filters
- Priority/status tabs
- Main queue table/list
- Detail drawer or detail route
- Bulk actions only where already approved
- Empty state with explanation

Used for:

- Moderation queue
- Grading tasks
- Review approvals
- Platform provisioning jobs
- Dead-letter/event review, where approved

4. Grid System

4.1 Base grid

Atlas uses a 12-column responsive grid.

Viewport	Grid
Mobile	4 columns
Tablet	8 columns
Desktop	12 columns
Wide desktop	12 columns with max-width container

4.2 Container widths

Container	Max Width	Usage
<code>container-public</code>	1120px	Public landing, verification
<code>container-learner</code>	1200px	Learner app pages
<code>container-control</code>	1440px	Studio/admin/moderation
<code>container-platform</code>	1600px	Platform console
<code>container-readable</code>	760px	Long-form lessons, policy copy
<code>container-form</code>	560px	Auth, reset, invite, basic forms

4.3 Column patterns

Pattern	Columns	Usage
Single column	12	Mobile, forms, lesson reading
Main + side	8 + 4	Detail pages, learner dashboards
Editor	3 + 6 + 3	Builders
Wide table	12	Admin/platform tables
Cards	3/4/6 responsive	Catalog, badges, certificates
Feed	8 + 4	Community, learner activity

4.4 Gutters

Viewport	Gutter
Mobile	16px
Tablet	20px
Desktop	24px
Wide	32px

5. Breakpoints

Use Tailwind defaults unless explicitly overridden.

```
screens: {  
  sm: "640px",  
  md: "768px",  
  lg: "1024px",
```

```
x1: "1280px",
"2x1": "1536px"
}
```

5.1 Responsive shell behavior

Shell	Mobile	Tablet	Desktop
Public	Single column	Single/two-column sections	Full hero/grid sections
Learner	Bottom nav or drawer	Collapsible sidebar	Persistent sidebar
Studio	Drawer nav + stacked panels	Collapsible panels	Three-pane builder
Moderation	Queue first, detail below	Queue + drawer detail	Queue + detail split
Tenant Admin	Drawer nav	Collapsible nav	Persistent sidebar
Platform	Drawer nav with warning	Collapsible nav	Persistent platform nav

5.2 Mobile cut rules

On mobile:

- No horizontal table overflow without explicit scroll container.
- Primary actions move to bottom sticky bar only when not covering content.
- Side panels become accordions or drawers.
- Builder right panels become "Settings" drawer.
- Builder left outline becomes "Outline" drawer.
- Charts must have text summaries above or below the visualization.
- Community composer becomes full-width modal/drawer.

6. Typography System

6.1 Font strategy

Use system font stack by default:

```
font-family: Inter, ui-sans-serif, system-ui, -apple-system,
BlinkMacSystemFont, "Segoe UI", sans-serif;
```

Tenant custom fonts are allowed only if the tenant branding configuration provides approved font references. Font loading must not block route gates.

6.2 Type scale

Token	Size	Line Height	Usage
<code>text-display</code>	48px	56px	Public hero only
<code>text-h1</code>	36px	44px	Page-level marketing heading
<code>text-h2</code>	30px	38px	Section heading
<code>text-h3</code>	24px	32px	Page title / card section
<code>text-h4</code>	20px	28px	Panel heading
<code>text-body-lg</code>	18px	28px	Public intro, lesson lead
<code>text-body</code>	16px	24px	Default text
<code>text-body-sm</code>	14px	20px	Secondary text, tables
<code>text-caption</code>	12px	16px	Metadata, badges, helper text
<code>text-micro</code>	11px	14px	Dense audit metadata only

6.3 Font weights

Token	Weight	Usage
<code>font-regular</code>	400	Body
<code>font-medium</code>	500	Labels, nav
<code>font-semibold</code>	600	Card titles, table headers
<code>font-bold</code>	700	Major headings

6.4 Typography rules

- Page titles use `text-h3` in app/control planes.
- Public landing hero may use `text-display`.
- Tables default to `text-body-sm`.
- Labels must not rely on placeholder text.
- Status labels use `text-caption` or `text-body-sm`.
- Legal/disclaimer copy uses `text-body-sm` minimum, never micro text.

7. Spacing System

7.1 Base scale

Use an 8px spacing system.

Token	Value
space-0	0
space-1	4px
space-2	8px
space-3	12px
space-4	16px
space-5	20px
space-6	24px
space-8	32px
space-10	40px
space-12	48px
space-16	64px
space-20	80px
space-24	96px

7.2 Page spacing

Context	Page Padding
Public mobile	16px
Public desktop	32–48px
Learner mobile	16px
Learner desktop	24px
Control mobile	16px
Control desktop	24–32px
Platform desktop	24px

7.3 Component spacing

Component	Internal Padding
Small button	8px 12px
Medium button	10px 16px
Large button	12px 20px
Card	20px

Component	Internal Padding
Dense card	16px
Modal	24px
Table cell	12px 16px
Compact table cell	8px 12px

8. Iconography Rules

8.1 Icon library

Use `lucide-react` as the default icon set because it aligns with shadcn/ui.

8.2 Icon sizes

Token	Size	Usage
<code>icon-xs</code>	12px	Dense metadata
<code>icon-sm</code>	16px	Inline labels, table rows
<code>icon-md</code>	20px	Buttons, nav
<code>icon-lg</code>	24px	Cards, empty states
<code>icon-xl</code>	32px	Public/empty illustrations

8.3 Icon rules

- Icons must not be the only indicator of meaning.
- Destructive icons require text labels.
- Navigation icons must be paired with labels on desktop.
- Mobile nav icons may use text below the icon.
- Status icons must pair with a badge or text.
- Platform admin icons must stay neutral and operational.

8.4 Approved semantic icon patterns

Meaning	Icon Direction
Learning	Book, play, file-text
Assessment	Clipboard, check-circle, timer
Practice/swipe	Layers, zap, repeat
Certificate	Award, badge-check

Meaning	Icon Direction
Community	Message-circle, users
Moderation	Shield, flag
Admin	Settings, sliders
Platform	Server, building, activity
Audit	History, scroll-text
Warning	Triangle-alert
Locked/permission	Lock
Entitlement	Sparkles or badge, paired with upgrade text

9. Color Token System

9.1 Token philosophy

Atlas separates:

1. Base UI tokens
2. Semantic tokens
3. Tenant brand tokens
4. Role/surface tokens
5. Chart tokens

Tenant branding may influence primary/accent tokens, but must never break contrast, readability, destructive semantics, or platform-admin neutrality.

9.2 Core shadcn-compatible tokens

```
:root {
  --background: 0 0% 100%;
  --foreground: 222 47% 11%;

  --card: 0 0% 100%;
  --card-foreground: 222 47% 11%;

  --popover: 0 0% 100%;
  --popover-foreground: 222 47% 11%;

  --primary: var(--tenant-primary, 222 47% 11%);
  --primary-foreground: var(--tenant-primary-foreground, 0 0% 100%);

  --secondary: 210 40% 96%;
  --secondary-foreground: 222 47% 11%;
```

```

--muted: 210 40% 96%;
--muted-foreground: 215 16% 47%;

--accent: var(--tenant-accent, 210 40% 96%);
--accent-foreground: var(--tenant-accent-foreground, 222 47% 11%);

--destructive: 0 84% 60%;
--destructive-foreground: 0 0% 100%;

--border: 214 32% 91%;
--input: 214 32% 91%;
--ring: var(--tenant-primary, 222 47% 11%);

--radius: 0.75rem;
}

```

9.3 Dark mode tokens

```

.dark {
  --background: 222 47% 8%;
  --foreground: 210 40% 98%;

  --card: 222 47% 10%;
  --card-foreground: 210 40% 98%;

  --popover: 222 47% 10%;
  --popover-foreground: 210 40% 98%;

  --secondary: 217 32% 17%;
  --secondary-foreground: 210 40% 98%;

  --muted: 217 32% 17%;
  --muted-foreground: 215 20% 65%;

  --border: 217 32% 18%;
  --input: 217 32% 18%;
}

```

9.4 Tenant brand tokens

```

:root {
  --tenant-primary: 222 47% 11%;
  --tenant-primary-foreground: 0 0% 100%;
  --tenant-accent: 210 40% 96%;
  --tenant-accent-foreground: 222 47% 11%;
  --tenant-logo-bg: transparent;
  --tenant-header-bg: var(--background);
}

```

```
--tenant-header-fg: var(--foreground);
}
```

9.5 Surface tokens

```
:root {
  --surface-public: var(--background);
  --surface-learner: var(--background);
  --surface-control: var(--background);
  --surface-platform: 222 47% 7%;

  --surface-platform-foreground: 210 40% 98%;
  --surface-warning: 45 93% 47%;
  --surface-success: 142 71% 45%;
  --surface-info: 217 91% 60%;
}
```

10. Semantic Color Usage

10.1 Semantic colors

Semantic	Usage
Primary	Main tenant-branded action
Secondary	Secondary actions, neutral panels
Muted	Metadata, subdued UI, placeholders
Accent	Tenant accent highlights, selected states
Destructive	Delete, revoke, remove, suspend, irreversible actions
Warning	Risk, review needed, incomplete setup
Success	Completed, active, verified, passed
Info	Informational banners, neutral guidance
Border	Dividers, cards, table rows
Ring	Focus states

10.2 State color rules

State	Visual Treatment
Active	Success badge
Invited	Warning/neutral badge

State	Visual Treatment
Suspended	Destructive or warning badge depending severity
Removed	Muted/destructive badge
Draft	Muted badge
Review	Warning badge
Published	Success badge
Revoked	Destructive badge
Failed	Destructive badge
Completed	Success badge
Locked	Muted badge with lock icon
Entitlement Required	Accent/info panel with upgrade/admin wording

10.3 Color safety

- Do not use color alone to communicate status.
- Always pair semantic colors with label text.
- Maintain WCAG AA contrast.
- Tenant-provided colors must be normalized into accessible foreground/background pairs.
- Platform console must not inherit tenant primary color for shell-level UI.

11. Component Library

11.1 Component layers

Atlas frontend components are grouped into four layers.

```
components/
  ui/           shadcn base components
  primitives/   Atlas wrappers around ui
  patterns/     reusable product patterns
  domains/      LMS/assessment/community/admin-specific components
```

11.2 Base UI components

Use shadcn/ui for:

- Button
- Input
- Textarea
- Select

- Checkbox
- Radio Group
- Switch
- Dialog
- Drawer
- Sheet
- Popover
- Tooltip
- Dropdown Menu
- Command
- Tabs
- Accordion
- Card
- Badge
- Alert
- Table
- Skeleton
- Avatar
- Separator
- Progress
- Toast/Sonner
- Form
- Calendar, only where existing date workflows require it

11.3 Atlas primitives

Component	Purpose
AtlasButton	Permission-aware button wrapper
AtlasBadge	Status/semantic badge
AtlasCard	Standard card wrapper
AtlasInput	Form input wrapper
AtlasSelect	Select wrapper
AtlasEmptyState	Standard empty state
AtlasSkeleton	Standard loading state
AtlasAlert	Error/info/warning/success alert
AtlasTable	Data table shell
AtlasPageHeader	Standard page header
AtlasDrawer	Mobile/detail drawers
AtlasDialog	Modal standard
AtlasTabs	Tab standard
AtlasProgress	Progress bars

Component	Purpose
AtlasAvatar	Member avatar

11.4 Product patterns

Pattern	Purpose
GuardedAction	Button/action rendered only after permission/entitlement resolution
PermissionBoundary	Safe denied/fallback rendering
EntitlementBoundary	Upgrade/locked-state rendering
ResourceStatusBadge	DB/API state display
AuditTrailInline	Sensitive action context
ReviewGatePanel	Human workflow state and next action
IdempotentSubmitButton	Prevent duplicate mutating requests
ConfirmDestructiveDialog	Required destructive confirmation
TenantBrandHeader	Tenant logo/theme header
SafeExternalCta	Attributed external CTA with legal copy
ScoreSummaryCard	Score/band display with text fallback
MetricCard	Dashboard metric
ChartCard	Analytics chart container
FilterBar	Search/filter/sort controls
BulkActionBar	Only where approved by source screens
MobileActionBar	Safe sticky mobile primary actions

11.5 Domain components

Domain	Components
Learning	CourseCard, LessonPlayerFrame, PathStepCard, ProgressRing, LessonCompletionButton
Assessment	QuestionCard, AttemptTimer, AnswerOption, AttemptProgress, GradingRubricPanel, AttemptResultCard
Swipe	SwipeCard, SwipeChoiceControls, SwipeProgress, SwipeSessionSummary
Competency	CompetencyBar, DimensionScoreList, ReadinessBandCard, ReadinessDisclaimer

Domain	Components
Certification	CertificateCard, CredentialVerifyCard, CertificateStatusBadge
Gamification	XpWidget, StreakWidget, BadgeGrid, LeaderboardTable
Community	PostCard, ThreadComposer, SpaceHeader, ModerationFlagButton, CommentList
Moderation	ModerationQueueTable, ModerationEvidencePanel, ModerationDecisionPanel, AppealReviewPanel
Admin	SetupChecklist, MemberRoleEditor, BrandingPreview, EntitlementReviewPanel, AuditLogTable
Platform	TenantListTable, TenantLifecyclePanel, ReasonScopeBanner, PlatformHealthCard

12. Form System

12.1 Form principles

Forms must be:

- Server-validated with Zod
- Client-assisted but never client-trusted
- Accessible by keyboard
- Label-first, not placeholder-first
- Error-specific
- Idempotency-safe where mutation requires it
- Permission-aware

12.2 Form anatomy

```
<FormShell>
  <FormHeader />
  <FormSection />
  <FormActions />
  <FormErrorSummary />
</FormShell>
```

12.3 Field anatomy

Each field must include:

- Visible label
- Optional description
- Input/control

- Inline validation error
- Optional helper text
- Required indicator where needed

12.4 Validation behavior

Error Type	UI Behavior
Field validation	Inline below field
Form-level validation	Error summary above actions
Auth error	Safe message; no account existence leak
Permission error	Permission denied state
Entitlement error	Entitlement required state
Conflict	Inline or form summary with safe retry
Rate limit	Banner with safe wait/retry copy
Server error	Non-sensitive alert with request ID

12.5 Form layout

Form Type	Layout
Auth/signup/reset	Single column, max 560px
Builder config	Right panel sections
Admin settings	Sectioned forms
Member/role forms	Detail page with side panel
Destructive forms	Modal with confirmation phrase only when needed
Platform reason forms	Required reason textarea, minimum length enforced by backend

12.6 Form actions

Action order:

1. Primary submit
2. Secondary save draft, if applicable
3. Cancel/back
4. Destructive action separated visually

Destructive actions never sit next to primary save without separation.

13. Table System

13.1 Table principles

Tables are used for admin, instructor, moderation, analytics, platform, audit, and resource management screens.

Every data table must support only filters/sorts declared by the API. Do not implement arbitrary client-side fields that imply unsupported backend filtering.

13.2 Table anatomy

```
<TablePage>
  <TableHeader />
  <FilterBar />
  <DataTable />
  <PaginationBar />
  <TableEmptyState />
</TablePage>
```

13.3 Table components

Component	Purpose
<code>DataTable</code>	Base table wrapper
<code>TableToolbar</code>	Search/filter/action row
<code>TableColumnHeader</code>	Sortable column label
<code>TableRowActions</code>	Row-level dropdown
<code>TableStatusCell</code>	Badge/status
<code>TableMemberCell</code>	Avatar/name/email-safe projection
<code>TableDateCell</code>	Relative + absolute date
<code>TablePagination</code>	Cursor pagination
<code>TableSkeletonRows</code>	Loading state
<code>TableEmptyState</code>	No rows state

13.4 Table rules

- Cursor pagination only.
- No offset pagination.
- Keep row actions in an overflow menu for dense tables.
- Keep destructive actions inside confirmation dialog.
- Do not expose raw global principal IDs.

- Use membership/profile identity projections only in tenant UI.
- Platform tables may show tenant IDs where approved by platform APIs.

13.5 Column density

Table Type	Density	Required Columns
Learner history	Standard	Name, status, progress/result, date
Instructor resources	Standard	Title, status, owner, updated, actions
Moderation queue	Compact	Priority, content type, reporter, status, age
Tenant admin members	Compact	Member, role, status, joined, actions
Audit logs	Compact	Time, actor, action, resource, request ID
Platform tenants	Compact	Tenant, state, domain, plan/entitlement summary, updated

14. Dashboard Components

14.1 Dashboard principles

Dashboards must answer: “What needs attention now?”

14.2 Dashboard component types

Component	Usage
<code>MetricCard</code>	Single number + delta/context
<code>ProgressSummaryCard</code>	Learner/course progress
<code>NextBestActionCard</code>	Learner next action
<code>SetupChecklist</code>	Tenant admin onboarding
<code>ReviewQueueCard</code>	Instructor/admin review count
<code>HealthStatusCard</code>	Platform/admin health
<code>RecentActivityList</code>	Activity stream
<code>RiskPanel</code>	Warnings and blocked setup
<code>ActionRequiredList</code>	Pending review/moderation/config work

14.3 Learner dashboard rules

Learner dashboard must prioritize:

1. Readiness band

2. Continue learning
3. Recommended practice
4. Streak/XP
5. Pending assessments/certificates
6. Community prompts, if entitled

14.4 Admin dashboard rules

Tenant admin dashboard must prioritize:

1. Setup checklist
2. Tenant health/config completeness
3. Membership state
4. Content review state
5. Entitlement/config warnings
6. Analytics summary, if entitled

14.5 Platform dashboard rules

Platform dashboard must prioritize:

1. Tenant lifecycle state
2. Provisioning health
3. Suspended/failed tenants
4. Audit/support activity
5. Queue/dead-letter health
6. Entitlement/config drift

15. Analytics Components

15.1 Analytics principles

Analytics must be readable, non-deceptive, and accessible. Charts must always include text summaries or tabular fallback.

15.2 Analytics components

Component	Purpose
<code>AnalyticsMetricCard</code>	KPI display
<code>TrendLineCard</code>	Time trend
<code>DistributionBarCard</code>	Score/progress distribution
<code>FunnelStepList</code>	Approved conversion/CTA funnel steps
<code>DimensionRadarFallback</code>	Text/table alternative for radar-like views

Component	Purpose
CohortTable	Learner/content cohort data
ItemStatsTable	Assessment item performance
ExportActionPanel	Export action where approved

15.3 Chart rules

- Do not use color alone.
- Always label axes.
- Always show timeframe.
- Always show empty/no-data state.
- Never imply unsupported causation.
- Readiness analytics must carry educational disclaimer where learner action is prompted.
- Dashboard widgets must not bypass entitlement gates.

15.4 Approved chart types

Chart	Use
Line	Trends
Bar	Comparisons
Stacked bar	Status distributions
Donut	Simple composition only, max 5 categories
Radar	Only when paired with accessible dimension list
Table	Audit, precise values, admin review

16. Assessment Components

16.1 Assessment principles

Assessment UI must be calm, legible, resilient, and explicit about submission state.

16.2 Assessment runner anatomy

```
<AssessmentRunner>
  <AttemptHeader />
  <AttemptProgress />
  <QuestionCard />
  <AnswerControls />
```

```
<AttemptFooterActions />
</AssessmentRunner>
```

16.3 Components

Component	Purpose
<code>AttemptHeader</code>	Title, timer, status
<code>AttemptProgress</code>	Current item/progress
<code>QuestionCard</code>	Prompt and media
<code>AnswerOption</code>	MCQ/choice response
<code>ShortAnswerInput</code>	Text response
<code>RubricPanel</code>	Grading rubric
<code>AttemptSubmitDialog</code>	Submit confirmation
<code>AttemptResultCard</code>	Result summary
<code>GradingTaskPanel</code>	Instructor grading
<code>IntegrityNotice</code>	Approved integrity/proctoring notice

16.4 Assessment rules

- “Submit” must confirm when irreversible.
- Autosave state must be visible where applicable.
- Timer must not be the only urgency indicator.
- Answer controls must be keyboard accessible.
- Results must show state, not just score.
- Grading UI must show assigned task context.
- Diagnostic and readiness assessments use generic assessment components with configuration.

17. Swipe Learning Components

17.1 Swipe principles

Swipe learning must be fast, mobile-first, and habit-oriented while remaining accessible without gesture-only interaction.

17.2 Swipe runner anatomy

```
<SwipeSession>
  <SwipeSessionHeader />
  <SwipeProgress />
```

```

<SwipeCard />
<SwipeChoiceControls />
<SwipeFeedback />
<SwipeSessionSummary />
</SwipeSession>

```

17.3 Components

Component	Purpose
<code>SwipeCard</code>	Prompt card
<code>SwipeChoiceControls</code>	Explicit left/right or true/false buttons
<code>SwipeKeyboardHints</code>	Keyboard alternatives
<code>SwipeProgress</code>	Session progress
<code>SwipeFeedback</code>	Correct/incorrect/explanation
<code>SwipeSessionSummary</code>	Session result
<code>SwipeRetryAction</code>	Repeat session where approved
<code>SwipeSignalNotice</code>	Explains that results affect competency only after server validation

17.4 Swipe rules

- Gesture must have button equivalent.
- Keyboard controls must exist.
- Feedback should be immediate but not distracting.
- Offline or batched responses must show pending validation state.
- Readiness impact must be explained as educational signal, not guarantee.
- Do not create a separate FundedBeyond-only swipe UI.

18. Community Components

18.1 Community principles

Community UI must support participation, moderation, privacy, and entitlement gates. There is no public community browsing in this phase.

18.2 Components

Component	Purpose
<code>CommunityHub</code>	Space overview
<code>SpaceCard</code>	Space summary

Component	Purpose
SpaceHeader	Space title, membership/status
PostComposer	Create post
PostCard	Feed item
PostDetail	Thread view
CommentList	Comments
ReportContentAction	Report content
ModerationStatusBadge	Moderation state
CommunityEmptyState	No posts/spaces state

18.3 Community rules

- Private spaces require membership checks.
- Report action must be visible where allowed.
- Moderator actions must be separated from normal member actions.
- Deleted/hidden content must show safe placeholder where needed.
- Empty community states should prompt permitted actions only.
- Community nav is hidden when entitlement is absent.

19. Admin Components

19.1 Admin principles

Tenant admin UI must be operational, safe, and audit-aware.

19.2 Components

Component	Purpose
AdminSetupChecklist	Setup status
MemberTable	Tenant members
MemberDetailPanel	Profile/roles/status
RoleAssignmentEditor	Role changes
PermissionOverridePanel	Overrides where permitted
BrandingEditor	Branding/theme config
BrandPreview	Preview tenant theme
DomainStatusPanel	Domain verification

Component	Purpose
EntitlementReviewPanel	Entitlement state
AuditLogTable	Audit history
DataExportPanel	Export request/status
DeletionRequestPanel	Deletion request/status
NotificationTemplateEditor	Notification template config
LocaleConfigPanel	Locale settings

19.3 Admin rules

- Sensitive actions show confirmation.
- Role/permission changes show audit impact.
- Owner-only actions are shown only after permission check.
- Entitlement-disabled areas preserve data and show upgrade/admin message.
- Admin pages must never expose platform actions.
- Branding preview must use tenant data only.
- Domain actions must clearly show status and next step.

20. Platform Components

20.1 Platform principles

Platform UI must be physically and visually isolated from tenant UI.

20.2 Components

Component	Purpose
PlatformReasonBanner	Shows current reason scope
PlatformTenantTable	Cross-tenant tenant list
TenantLifecyclePanel	Provision/suspend/reactivate/archive controls
PlatformEntitlementPanel	Tenant entitlements
ProvisioningJobTable	Provisioning jobs
PlatformAuditViewer	Cross-tenant/platform audit
SupportScopePanel	Reason-bound tenant support context
PlatformHealthCard	System status
DeadLetterQueueTable	Event/worker issue review, where approved

20.3 Platform rules

- Platform shell must not inherit tenant branding.
 - Every support/lifecycle action must show reason context.
 - Reason must be visible while scoped access is active.
 - Cross-tenant actions must show tenant identity clearly.
 - Destructive lifecycle actions require confirmation.
 - Platform pages must never embed tenant app screens.
 - Platform support access must feel exceptional, not normal.
-

21. Modal System

21.1 Modal types

Modal Type	Component	Usage
Informational	<code>InfoDialog</code>	Explain state
Confirmation	<code>ConfirmDialog</code>	Safe confirmation
Destructive	<code>ConfirmDestructiveDialog</code>	Delete/revoke/suspend/remove
Form	<code>FormDialog</code>	Small scoped forms
Drawer	<code>DetailDrawer</code>	Mobile/detail panels
Identity Gate	<code>IdentityGateDialog</code>	Diagnostic signup gate
MFA	<code>MfaDialog</code>	Admin/platform MFA flow
Upgrade/Entitlement	<code>EntitlementRequiredDialog</code>	Entitlement blocked action

21.2 Modal rules

- Trap focus.
 - Return focus to opener.
 - Close on Escape unless unsafe.
 - Destructive actions require explicit confirmation.
 - Do not use modals for full complex builders.
 - On mobile, large modals become drawers or full-screen sheets.
 - Modals must not bypass route gates.
-

22. Notification System

22.1 Notification surfaces

Surface	Usage
Toast	Immediate local feedback
Inline alert	Persistent contextual message
Notification center	User notifications
Banner	System/tenant-wide state
Email template preview	Admin notification config

22.2 Toast rules

Use toasts for:

- Saved
- Submitted
- Copied
- Sent
- Joined
- Completed
- Queued
- Retried

Do not use toasts as the only record of sensitive action success. Sensitive actions need visible page state or audit/history indicator.

22.3 Inline alert rules

Use inline alerts for:

- Permission limitations
- Entitlement required
- Review state
- Form-level errors
- Tenant unavailable
- Legal/readiness disclaimers
- Rate limits

22.4 Notification center rules

- Show unread state.
- Use safe previews.
- Do not leak restricted resource titles if permission changes.
- Clicking a notification must re-run route gates.

23. Empty States

23.1 Empty state anatomy

```
<EmptyState>
  <Icon />
  <Title />
  <Description />
  <PermittedPrimaryAction />
  <OptionalSecondaryAction />
</EmptyState>
```

23.2 Empty state rules

- Show one permitted primary action only.
- If the user cannot create/join/act, show explanatory read-only copy.
- Do not reveal hidden features.
- Do not show admin CTAs to learners.
- Do not show platform CTAs inside tenant UI.
- Use tenant branding lightly, not decorative overload.

23.3 Empty state examples

Context	Message Direction
No courses	"No courses are available yet."
No attempts	"You have not started an assessment yet."
No certificates	"Certificates you earn will appear here."
No community posts	"No posts yet in this space."
No moderation cases	"No cases need review."
No audit rows	"No audit events match these filters."
No platform tenants	"No tenants match the current filters."

24. Loading States

24.1 Loading principles

Loading states must be safe, non-leaky, and layout-stable.

24.2 Loading hierarchy

Stage	UI
Tenant resolving	Minimal neutral shell; no tenant data
Public data loading	Public skeleton with tenant-safe brand only after resolution
Auth/membership loading	Shell skeleton only
Entitlement/permission loading	No protected action rendered
Resource loading	Page body skeleton
Mutation loading	Disable action, show pending state
Background refresh	Subtle inline loading indicator

24.3 Skeleton rules

- Skeletons must match final layout dimensions.
- Do not skeleton protected nav before gate passes.
- Do not show fake rows with real-looking names.
- Keep skeleton animation subtle.
- On mobile, reduce skeleton complexity.

25. Error States

25.1 Error state principles

Errors must map to API error conventions and avoid tenant/resource leakage.

25.2 Error mapping

Code	UI
VALIDATION_ERROR	Inline field/form errors
AUTH_REQUIRED	Login redirect or auth-required state
NO_MEMBERSHIP	No-access membership state
MEMBERSHIP_PENDING	Invitation acceptance guidance
MEMBERSHIP_SUSPENDED	Suspended membership notice
MEMBERSHIP_REMOVED	Removed membership notice
PERMISSION_DENIED	Permission denied state
OWNERSHIP_DENIED	Permission denied state, no resource leak

Code	UI
RELATIONSHIP_DENIED	Permission denied state, no resource leak
ENTITLEMENT_REQUIRED	Entitlement required state
TENANT_NOT_FOUND	Tenant unavailable/not found
TENANT_SUSPENDED	Tenant unavailable/suspended
CONFLICT	Conflict resolution message
IDEMPOTENCY_REPLAY	Safe “already processed” state
RATE_LIMITED	Rate-limit alert
SERVER_ERROR	Generic error with request ID

25.3 Error component

```
<ErrorState
  code="PERMISSION_DENIED"
  title="You do not have access"
  description="Your current role does not allow this action."
  requestId={requestId}
/>
```

25.4 Error rules

- Always show safe human message.
- Show request ID for support.
- Never reveal cross-tenant resource existence.
- Never show stack traces.
- Never expose raw database errors.
- Do not show upgrade prompt for pure permission denial.

26. Permission Denied States

26.1 Permission denied principles

Permission denied means the actor is known but not allowed to perform the action.

26.2 UI patterns

Context	Pattern
Whole route denied	Full-page denied state
Action denied	Hide action or disabled action with explanation

Context	Pattern
Field denied	Read-only field
Owned resource denied	Safe denied state without owner/resource leak
Relationship denied	Safe denied state without learner/course leak
Platform denied	Platform denied state with no tenant data

26.3 Copy standard

Title: "You do not have access"

Description: "Your current role does not allow this action in this tenant."

For ownership/relationship denial, do not say "this belongs to another user" if it could leak existence.

26.4 Rules

- Do not use entitlement upgrade UI for permission failure.
- Do not expose hidden nav.
- Do not show disabled admin controls to learners.
- Do not suggest contacting platform support unless the support path is already approved.
- Preserve route safety over helpfulness.

27. Entitlement Required States

27.1 Entitlement principles

Entitlement required means the tenant plan/configuration does not include the capability. It is not an actor-specific role denial.

27.2 UI patterns

Context	Pattern
Route-level entitlement missing	Full-page entitlement required
Sub-action entitlement missing	Locked card or disabled action
Admin entitlement review	Entitlement review panel
Existing data	Read-only preservation state where applicable

27.3 Copy standard

Title: "This feature is not enabled for this tenant"

Description: "Your tenant's current entitlements do not include this capability."

For admins/owners, include: "Review entitlements or contact the platform operator."

For learners/instructors/moderators, include: "Contact your academy admin if you believe this should be available."

27.4 Rules

- Do not delete or hide existing data from admin/export views.
 - Hide entitlement-gated nav items where route is unavailable.
 - Use `ENTITLEMENT_REQUIRED` state consistently.
 - Never check plan names in frontend logic.
 - Frontend may use server-provided entitlement projection only.
-

28. Accessibility Standards

28.1 Standard

Atlas UI must meet WCAG 2.2 AA for Phase 0 + Phase 1A + Phase 1B.

28.2 Minimum requirements

- Keyboard navigable
- Visible focus states
- Semantic HTML
- Accessible labels
- Error messages linked to inputs
- Dialog focus trap
- Escape/close semantics
- Sufficient color contrast
- Non-color-only status indicators
- Text fallback for charts
- Touch targets minimum 44px where practical
- Reduced motion support
- Screen-reader live regions for async errors/success

28.3 Chart accessibility

Charts must include:

- Title
- Description
- Timeframe
- Legend
- Text summary
- Table fallback for critical values

28.4 Assessment accessibility

Assessment and swipe components must support:

- Keyboard answer selection
 - Screen-reader prompt reading
 - Visible selected state
 - Non-gesture fallback
 - Timer text announcement without excessive interruption
 - Submit confirmation
-

29. Keyboard Navigation Standards

29.1 Global order

Focus order:

1. Skip link
2. Header
3. Navigation
4. Page title
5. Primary content
6. Secondary panels
7. Actions

29.2 Required shortcuts

Do not introduce global keyboard shortcuts that conflict with browser/screen-reader defaults.

Allowed scoped shortcuts:

Context	Shortcut
Dialog	Escape closes when safe
Select/command	Arrow navigation
Tabs	Arrow navigation
Swipe	Left/right or explicit key mapping with visible hint
Table	Tab through row actions
Builder panels	Tab order follows visual order

29.3 Focus management

- Route changes focus page title.
- Modal open focuses first meaningful element.

- Modal close returns focus to opener.
 - Error summary receives focus after failed submit.
 - Destructive confirmation focuses cancel by default unless design requires otherwise.
 - Toasts do not steal focus.
-

30. Responsive Design Standards

30.1 General rules

- Mobile-first CSS.
- No horizontal page overflow.
- Tables use responsive containers.
- Side panels become drawers/accordions.
- Sticky actions must not hide content.
- Learner screens prioritize single-column.
- Builder screens preserve outline/editor/settings via drawers on mobile.
- Platform console remains usable but optimized for desktop.

30.2 Mobile navigation

Plane	Mobile Nav
Public	Topbar menu
Learner	Bottom nav or drawer
Studio	Drawer
Moderation	Drawer
Tenant Admin	Drawer
Platform	Drawer with platform warning

30.3 Responsive table behavior

Mobile tables may convert to cards only when columns are not audit-critical.

Do not card-convert audit logs or platform-sensitive tables if exact row/column scanning is required. Use horizontal scroll with sticky first column where needed.

31. Dark Mode Standards

31.1 Dark mode support

Dark mode must be supported at token level for all shared components.

31.2 Rules

- Tenant branding must be contrast-corrected in dark mode.
- Logos must support dark/light variants where configured.
- Charts must use dark-safe color tokens.
- Status colors must remain readable.
- Platform shell remains visually distinct in both modes.
- Do not invert uploaded images automatically.
- Certificate/public verification must remain printable/readable in light mode.

31.3 Theme source

Theme mode may be:

- System
- User preference
- Tenant default, where configured

Frontend must not hardcode tenant-specific mode.

32. Tenant Branding Rules

32.1 Branding inputs

Allowed tenant branding inputs:

- Logo
- Favicon
- Primary color
- Accent color
- Header color
- Brand typography, if configured
- Domain/subdomain
- Public landing content
- Email-safe branding tokens

32.2 Branding outputs

Branding may affect:

- Public shell
- Learner shell
- Tenant admin preview
- Email template preview
- Certificate verification issuer display
- CTAs
- Selected states
- Header logo area

Branding must not affect:

- Destructive color semantics
- Permission denied semantics
- Platform shell
- Audit severity colors
- Error states
- Accessibility requirements

32.3 Branding preview

Branding preview must show:

- Header
- Button
- Card
- Form field
- Badge
- Alert
- Certificate/verification sample, if relevant
- Light/dark preview, if supported

Preview must clearly indicate whether changes are draft or published.

33. White-Label Rules

33.1 White-label invariant

Atlas must appear as infrastructure, not as the tenant's public identity, except where plan/entitlement intentionally allows or requires platform attribution.

33.2 Rules

- Public pages show tenant brand.
- Learner app shows tenant brand.
- Emails use tenant brand tokens.
- Certificates show tenant/issuer brand where configured.
- Platform console shows Atlas operational identity, not tenant brand.
- Tenant admin may see Atlas controls where needed.
- No "FundedBeyond" strings in shared components.
- No tenant-specific CSS files.
- No tenant-specific component forks.
- No tenant-specific route branches.

33.3 White-label implementation

Use:

```
<TenantThemeProvider tenantTheme={theme}>
  <TenantBrandProvider branding={branding}>
    {children}
  </TenantBrandProvider>
</TenantThemeProvider>
```

Do not use:

```
if (tenant.slug === "fundedbeyond") {
  // forbidden
}
```

34. Motion & Animation Rules

34.1 Motion principles

Motion should clarify state, not entertain.

34.2 Durations

Token	Duration	Usage
<code>motion-fast</code>	100ms	Hover/focus
<code>motion-base</code>	150ms	Button/menu
<code>motion-medium</code>	200ms	Drawer/modal
<code>motion-slow</code>	300ms	Page-level transitions

34.3 Easing

```
--ease-standard: cubic-bezier(0.2, 0, 0, 1);
--ease-emphasized: cubic-bezier(0.2, 0, 0, 1);
```

34.4 Rules

- Respect `prefers-reduced-motion`.
- Do not animate critical error text in a way that reduces readability.
- Swipe card motion must have reduced-motion fallback.
- Loading skeleton motion must be subtle.
- Platform console motion must be minimal.
- Modals/drawers use consistent enter/exit transitions.
- Do not use gamified confetti for sensitive assessment/certification outcomes unless explicitly configured and accessible.

35. Design Tokens

35.1 Token categories

```
tokens/  
  color  
  typography  
  spacing  
  radius  
  shadow  
  z-index  
  motion  
  layout  
  component  
  tenant
```

35.2 Radius tokens

Token	Value
<code>radius-sm</code>	6px
<code>radius-md</code>	8px
<code>radius-lg</code>	12px
<code>radius-xl</code>	16px
<code>radius-2xl</code>	24px
<code>radius-full</code>	9999px

35.3 Shadow tokens

Token	Usage
<code>shadow-xs</code>	Subtle cards
<code>shadow-sm</code>	Dropdowns
<code>shadow-md</code>	Modals/drawers
<code>shadow-lg</code>	Public hero cards only
<code>shadow-none</code>	Dense admin/platform

35.4 Z-index tokens

Token	Value
<code>z-base</code>	0
<code>z-header</code>	40
<code>z-sticky</code>	50
<code>z-drawer</code>	60
<code>z-popover</code>	70
<code>z-modal</code>	80
<code>z-toast</code>	90
<code>z-debug</code>	100

35.5 Tailwind token extension

```
export const atlasTheme = {
  borderRadius: {
    sm: "var(--radius-sm)",
    md: "var(--radius-md)",
    lg: "var(--radius-lg)",
    xl: "var(--radius-xl)",
  },
  spacing: {
    18: "4.5rem",
    22: "5.5rem",
  },
  colors: {
    border: "hsl(var(--border))",
    input: "hsl(var(--input))",
    ring: "hsl(var(--ring))",
    background: "hsl(var(--background))",
    foreground: "hsl(var(--foreground))",
    primary: {
      DEFAULT: "hsl(var(--primary))",
      foreground: "hsl(var(--primary-foreground))",
    },
    destructive: {
      DEFAULT: "hsl(var(--destructive))",
      foreground: "hsl(var(--destructive-foreground))",
    },
  },
};
```

36. shadcn Component Mapping

Atlas Need	shadcn Component	Atlas Wrapper
Buttons/ actions	Button	AtlasButton , GuardedAction
Cards	Card	AtlasCard , MetricCard
Forms	Form , Input , Textarea , Select	AtlasFormField
Dialogs	Dialog , AlertDialog	AtlasDialog , ConfirmDestructiveDialog
Drawers/ sheets	Sheet , Drawer	DetailDrawer , MobilePanel
Menus	DropdownMenu	ActionMenu
Search command	Command	GlobalSearchCommand
Tabs	Tabs	AtlasTabs
Accordions	Accordion	SettingsAccordion
Tables	Table	DataTable
Badges	Badge	StatusBadge
Alerts	Alert	InlineAlert
Skeletons	Skeleton	PageSkeleton , TableSkeletonRows
Progress	Progress	AttemptProgress , PathProgress
Tooltips	Tooltip	HelpTooltip
Toasts	Sonner	AtlasToastProvider
Avatars	Avatar	MemberAvatar

36.1 shadcn usage rules

- Do not modify shadcn source directly for product logic.
- Wrap shadcn components in Atlas primitives where permission, entitlement, status, analytics, or tenant theming is needed.
- Keep base `ui/` components close to shadcn defaults.
- Product-specific props belong in `patterns/` or `domains/`.

37. Component Naming Convention

37.1 Naming rules

Use PascalCase for React components.

```
[Domain][Purpose][Type]
```

Examples:

- LearnerDashboardPage
- CourseCard
- AssessmentRunner
- SwipeSessionSummary
- ModerationDecisionPanel
- TenantLifecyclePanel
- PlatformReasonBanner

37.2 File naming

Use kebab-case for filenames.

```
course-card.tsx  
assessment-runner.tsx  
platform-reason-banner.tsx
```

37.3 Folder convention

```
components/  
  ui/  
  primitives/  
  patterns/  
  domains/  
    learning/  
    assessment/  
    swipe/  
    competency/  
    certification/  
    gamification/  
    community/  
    moderation/  
    admin/  
    platform/  
app/  
  (public)/  
  (learner)/
```

```
studio/  
moderate/  
admin/  
platform/
```

37.4 Prop naming

Prop	Usage
<code>status</code>	Resource state
<code>variant</code>	Visual variant
<code>size</code>	Size
<code>permission</code>	Declared permission key when used by boundary
<code>entitlement</code>	Declared entitlement key when used by boundary
<code>resourceRef</code>	Resource authorization reference
<code>isLoading</code>	Loading state
<code>isDisabled</code>	Disabled state
<code>onAction</code>	Action callback
<code>requestId</code>	Error/support trace

38. Figma Structure

38.1 Figma file organization

```
Atlas LMS Design System v1  
  00 Cover & Scope  
  01 Foundations  
  02 Tokens  
  03 Typography  
  04 Color  
  05 Layout & Grid  
  06 Components - Base  
  07 Components - Patterns  
  08 Components - Learning  
  09 Components - Assessment & Swipe  
  10 Components - Community  
  11 Components - Admin  
  12 Components - Platform  
  13 States  
  14 Responsive
```

15 Screen Compositions
16 Handoff Notes

38.2 Figma component naming

Component / Domain / Name / Variant

Examples:

- Button / Primary / Medium
- Badge / Status / Published
- Card / Dashboard / Metric
- Learning / CourseCard / Default
- Assessment / QuestionCard / MultipleChoice
- Admin / MemberRoleEditor / Default
- Platform / ReasonBanner / Active

38.3 Figma variant properties

Use:

- variant
- size
- state
- density
- theme
- role
- status

38.4 Handoff rules

Every component must include:

- Purpose
 - Usage
 - Props/states
 - Accessibility notes
 - Responsive behavior
 - Token references
 - shadow mapping
 - Do/don't examples
-

39. Frontend Architecture Guidelines

39.1 App Router groups

Use route groups to isolate shells.

```
app/  
  (public)/  
  (learner)/  
  studio/  
  moderate/  
  admin/  
  platform/
```

39.2 Server-first routing

Protected pages should resolve gates server-side before rendering page UI.

```
export default async function Page() {  
  const ctx = await requireTenantActor({  
    permission: "course.read",  
    entitlement: undefined,  
  });  
  
  const data = await getAuthorizedPageData(ctx);  
  
  return <CourseCatalogPage data={data} />;  
}
```

39.3 Client component usage

Use client components only for:

- Interactive forms
- Menus
- Dialogs/drawers
- Swipe gestures
- Assessment controls
- Charts
- Rich builders
- Optimistic UI with rollback

Do not move authorization decisions into client components.

39.4 Data access rules

Frontend components consume API/resource-loader projections only. They do not query tables directly.

39.5 Permission boundaries

Use permission boundaries for display safety, not as the primary authorization mechanism.

```
<PermissionBoundary permission="course.update" resourceRef={course.ref}>
  <EditCourseButton />
</PermissionBoundary>
```

Server authorization remains authoritative.

39.6 Entitlement boundaries

```
<EntitlementBoundary entitlement="community.enable">
  <CommunityNavItem />
</EntitlementBoundary>
```

Entitlement boundaries must use server-provided entitlement projections and must not check plan names.

39.7 Mutation rules

All mutating actions must:

- Use declared API route or server action
- Include idempotency key where required
- Disable during submit
- Show safe success/error state
- Roll back optimistic UI on denial
- Surface request ID on error

39.8 Design-system package boundary

Recommended internal package structure:

```
packages/
  design-system/
    tokens/
    components/
    patterns/
  app-web/
  config/
```

Do not create tenant-specific design-system packages.

40. Major Screen Category Compositions

40.1 Learner

Page composition

```
LearnerShell
  LearnerHeader
    TenantLogo
    Search
    Notifications
    ProfileMenu
  LearnerNavigation
  LearnerMain
    PageHeader
    PrimaryLearningRegion
    SecondaryProgressRegion
```

Component hierarchy

```
LearnerDashboardPage
  ReadinessBandCard
  NextBestActionCard
  ContinueLearningCard
  StreakWidget
  XpWidget
  RecommendedPracticeCard
  PendingAssessmentCard
```

Reusable patterns

- Next best action
- Progress card
- Readiness band
- Swipe/practice prompt
- Lesson continuation
- Certificate card
- Community prompt
- Entitlement-gated widgets
- Ownership-scoped profile/settings

Design rules

- One primary next action per viewport.
- Roadmap/progress must be visually prominent.

- Swipe and diagnostic states must feel fast and mobile-first.
 - Do not overwhelm learners with admin-style analytics.
 - Readiness CTA must include educational disclaimer.
-

40.2 Instructor

Page composition

```
ControlShell
  ControlHeader
  StudioNav
  ControlMain
    PageHeader
    Builder/Table/DetailRegion
    ReviewStatusPanel
```

Component hierarchy

```
InstructorPage
  StudioResourceTable OR BuilderLayout
  ResourceStatusBadge
  GuardedPrimaryAction
  ReviewGatePanel
  ValidationPanel
  SaveStateIndicator
```

Reusable patterns

- Resource table
- Builder layout
- Draft/review/published status
- Submit for review
- Grading task panel
- Item stats panel
- Learner progress table, relationship-scoped
- Ownership-aware edit controls

Design rules

- Instructors see only owned/related resources unless admin role also applies.
 - Publish language must reflect workflow gate.
 - Validation and review comments must be visible near the action.
 - Builder UI must not create separate diagnostic-specific fork.
-

40.3 Moderator

Page composition

```
ControlShell
  ControlHeader
  ModerationNav
  ControlMain
    QueueFilters
    ModerationQueue
    CaseDetailPanel
```

Component hierarchy

```
ModerationPage
  ModerationQueueTable
  ModerationStatusBadge
  ModerationEvidencePanel
  ModerationDecisionPanel
  AppealReviewPanel
```

Reusable patterns

- Queue-first layout
- Evidence panel
- Decision panel
- Appeal panel
- Reported content safe preview
- Moderator-only actions
- Audit/history inline metadata

Design rules

- Evidence and context must be clear before decision actions.
- Destructive moderation actions require confirmation.
- Moderators do not get broad admin controls.
- Moderation history stays inside moderation context.
- Community entitlement gate applies.

40.4 Tenant Admin

Page composition

```
ControlShell
  ControlHeader
  AdminNav
  ControlMain
```

```
PageHeader
AdminPrimaryRegion
AdminSecondaryPanel
```

Component hierarchy

```
TenantAdminPage
  AdminSetupChecklist
  DataTable OR SettingsForm OR BuilderReuse
  EntitlementReviewPanel
  AuditInlinePanel
  GuardedPrimaryAction
```

Reusable patterns

- Setup checklist
- Member table/detail
- Role editor
- Permission override panel
- Branding/theme editor
- Domain status panel
- Entitlement review
- Notification template editor
- Audit log table
- Export/deletion panels
- Admin reuse of studio authoring screens

Design rules

- Owner-only actions live inside relevant screens, not separate screens.
- Admin can reuse Studio screens at tenant scope.
- Entitlement-disabled features preserve data and show clear state.
- Admin console must not expose platform routes.
- Audit-sensitive actions show confirmation and traceability.

40.5 Platform Admin

Page composition

```
PlatformShell
  PlatformHeader
    MFASStatus
    ReasonScopeBanner
  PlatformNav
  PlatformMain
    PlatformPageHeader
```

CrossTenantTableOrDetail
PlatformAuditPanel

Component hierarchy

PlatformPage
 PlatformReasonBanner
 PlatformTenantTable
 TenantLifecyclePanel
 PlatformEntitlementPanel
 ProvisioningJobTable
 PlatformAuditViewer
 PlatformHealthCard

Reusable patterns

- Reason-bound action
- Tenant lifecycle state
- Cross-tenant table
- Platform audit viewer
- Support scope panel
- Provisioning job table
- Health/dead-letter review
- Destructive lifecycle confirmation

Design rules

- Platform shell is visually isolated.
- No tenant branding in platform chrome.
- Every scoped tenant action shows reason context.
- Platform cannot look like tenant admin.
- Platform pages never embed tenant screens.
- All sensitive platform actions require visible audit context.

41. CTO Final Validation

41.1 Validation checklist

Area	Verdict
No new screens introduced	Pass
No new APIs introduced	Pass
No new permissions introduced	Pass
No new workflows introduced	Pass
No new entities introduced	Pass

Area	Verdict
No new roles introduced	Pass
Phase 0 + 1A + 1B only	Pass
FundedBeyond remains tenant configuration	Pass
Tenant branding is tokenized	Pass
Platform shell isolated	Pass
Gate-before-render preserved	Pass
Entitlement and permission states separated	Pass
shadcn/ui mapped without product fork	Pass
Tailwind token system defined	Pass
Accessibility standards defined	Pass
Responsive behavior defined	Pass
Dark mode standards defined	Pass
Figma structure defined	Pass
Frontend architecture aligned to Next.js App Router	Pass

41.2 Build gates

Before frontend implementation is accepted:

1. All shared components must use design tokens.
2. No tenant-specific component forks may exist.
3. No protected navigation may render before gate resolution.
4. All forms must have visible labels and Zod-backed validation.
5. All charts must have text/table fallback.
6. Permission denied and entitlement required states must be visually distinct.
7. Platform shell must not inherit tenant theme.
8. Destructive actions must use confirmation patterns.
9. Mobile layouts must pass no-horizontal-overflow checks.
10. Component names and Figma names must match this convention.
11. shadcn base components must remain separated from Atlas product wrappers.
12. Accessibility review must pass WCAG 2.2 AA for core flows.

41.3 Final verdict

ATLAS LMS — DESIGN SYSTEM v1 is approved as an implementation-ready design-system layer for Phase 0 + Phase 1A + Phase 1B.

It adds no product scope.

It does not redesign approved wireframes.

It does not alter architecture.

It does not modify permissions, APIs, workflows, roles, or entities.

It defines the visual, component, token, responsive, accessibility, and frontend implementation rules required to build the approved Atlas LMS screens consistently across learner, instructor, moderator, tenant admin, and platform admin planes.